

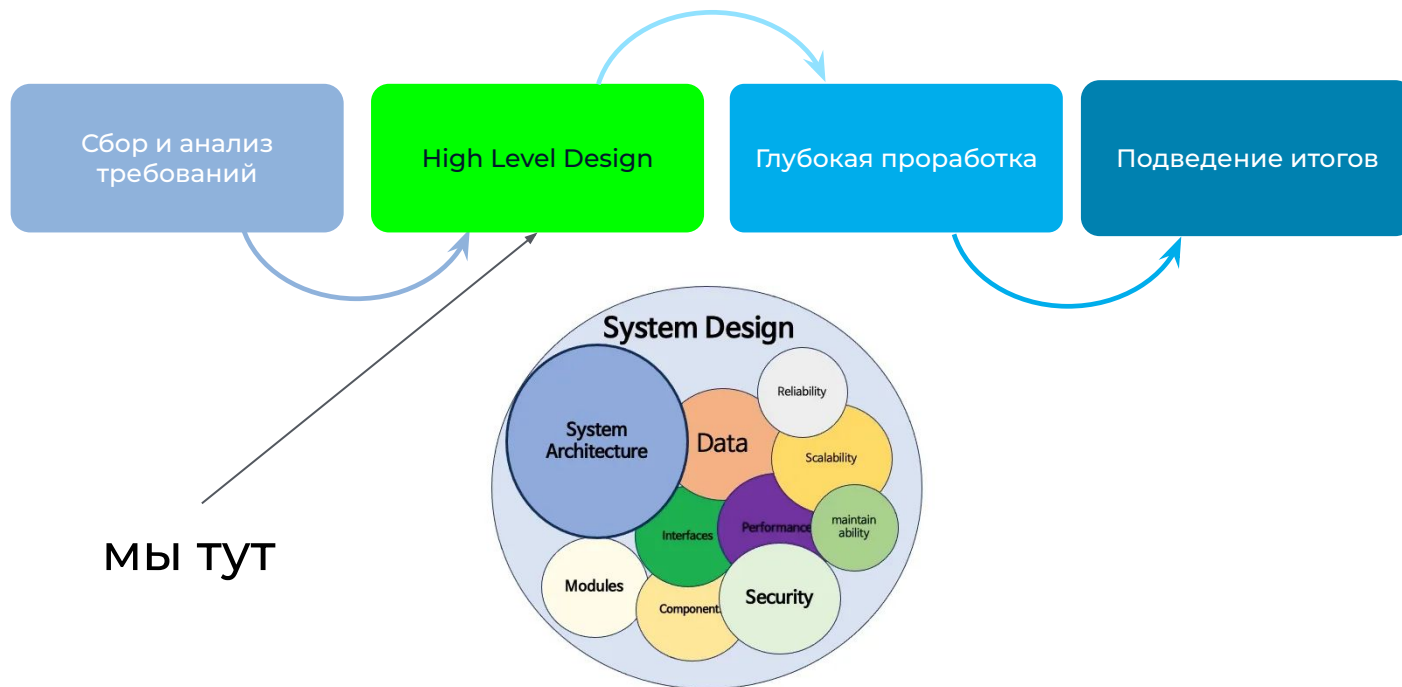


НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Системный дизайн современных приложений

Занятие №5
Архитектура. Часть №3
Интеграции.

Этапность





Ренессанс модульного монолита

Монолит

У нас монолитное приложение. Вроде вертикально масштабируемся. Сложно стало с разработкой, фичи выпускаем не так часто. Становится неуправляемым...

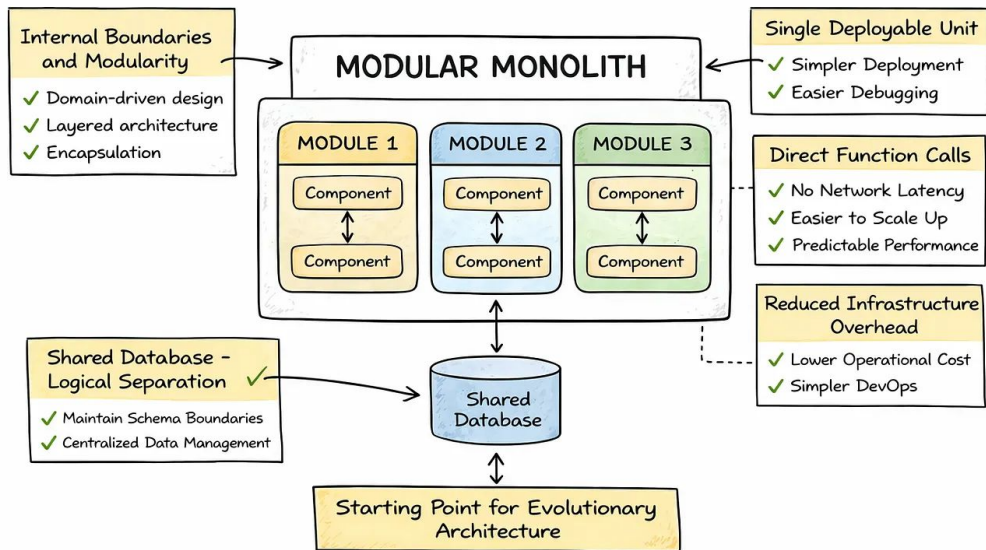
Микросервисы

Наш СТО услышал про Docker, какие-то новомодные микросервисы. Мы слышали что-то про SOA, давайте нарежем монолит на микросервисы!

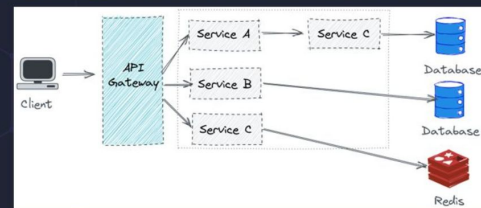
Распределенный монолит

Нарезали...

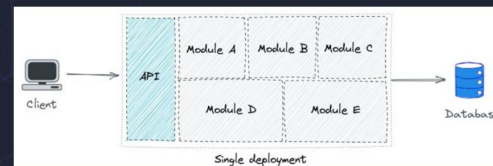
Ренессанс модульного монолита

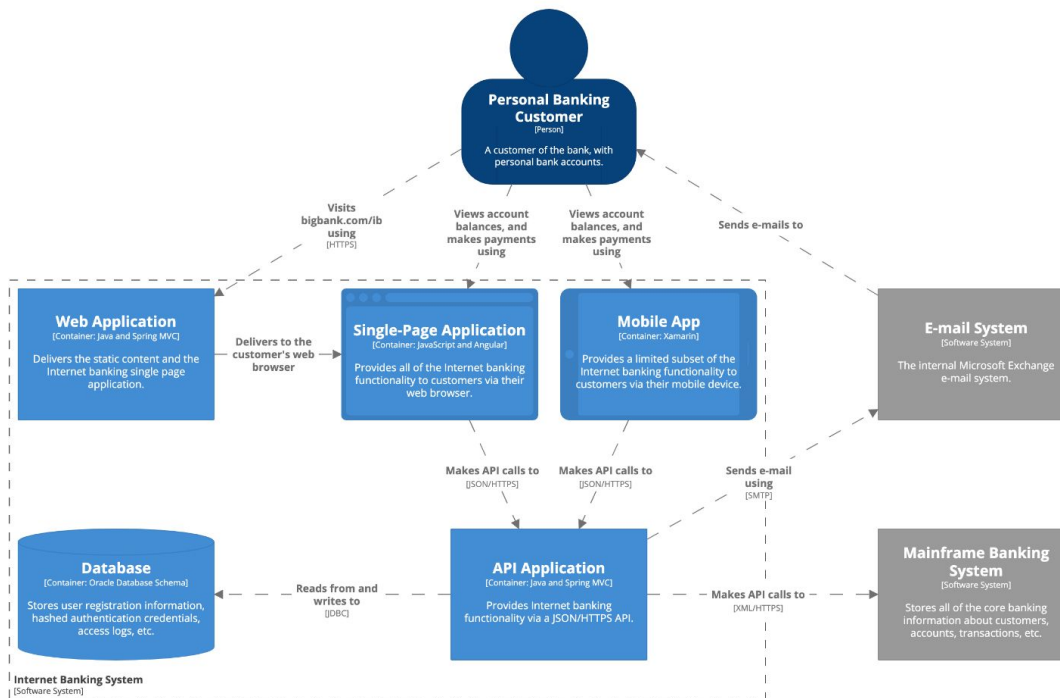


Shopify architecture is not



What it is...

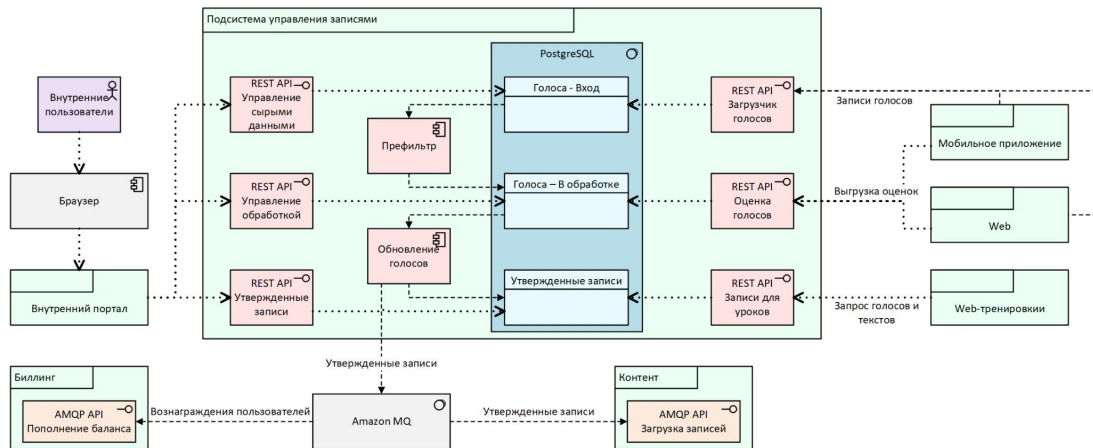
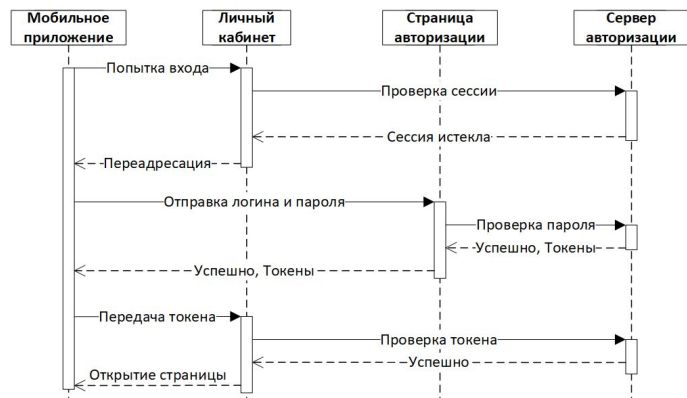




Container diagram for Internet Banking System

The container diagram for the Internet Banking System.

Workspace last modified: Wed Feb 05 2020 09:33:36 GMT+0100 (Central European Standard Time)





Язык программирования больше не важен

Но это не точно

Глобально, с приходом AI в SDLC (Software Development Life Cycle) - мы можем вообще ни разу не увидеть код, как это делают вайбкодеры

Особенности ЯП все равно важны в атрибутах качества системы

Все еще есть операционный опыт: дебажить Prod в 3 ночи, когда закончились токены...

LLM пока неравномерен: Python/TypeScript/Go - отлично знает; Erlang/Nim/Zig - значительно хуже



Just-in-Time архитектура

Если AI быстро пишет код, то можем быстро эволюционировать

Гипотеза: можем меньше принимать стратегических решений, т.к. тактически мы начинаем быстрее адаптироваться под изменение внешних условий, таким образом архитектура может перестраиваться по 3 раза в день

Но это работает для обратимых решений. Все еще важны необратимые: модель данных (миграции болезненны), публичные API контракты (клиенты уже интегрировались), выбор брокера сообщений (миграция данных), модель безопасности приложения



Spec-Driven Development

Раньше: архитектор -> диаграммы -> разработчики -> код

Сейчас: архитектор -> спецификации -> AI -> код -> разработчики проверяют, хотя можно настроить агента-валидатора и уйти в MAS

Спецификации становятся первичным артефактом. OpenAPI, ADR, архитектурные ограничения в машиночитаемом формате - AI это ест и генерирует реализацию

Что это значит для нас: навык писать точные, недвусмысленные спецификации становится важнее навыка писать код



Architecture as docs-as-code

Папка /architecture

/decisions: ADR файлы

/diagrams: C4 модели в коде (Structurizr DSL, Mermaid)

/constraints: ArchUnit правила

/specs: OpenAPI, AsyncAPI

CI/CD проверяет:

ADR существует для каждого значимого решения

диаграммы актуальны (сгенерированы из кода)

ArchUnit не нарушен



LLM знает только open source

Обучен на публичном коде (если не брать дообучение)

Знает хорошо:

- PostgreSQL, Kafka, Redis, Kubernetes, React, ...
- стандартные паттерны

Не знает:

- внутренние фреймворки компании
- проприетарные платформы
- легаси-системы без документации в интернете

Аргумент против самописных фреймворков и за стандартные решения



Самолечение архитектуры

Архитектурный уровень

AI-агент мониторит метрики архитектуры (связанность, зависимости, деградация производительности), обнаруживает сдвиг, предлагает рефакторинг или даже применяет его автоматически в пределах заданных границ



Все ресурсы у ИИ - что делать?

Стоит задуматься

Возможные тренды:

- Маленькие модели и edge-серверы: вычисления ближе к пользователю
- AI как инфраструктура: вместо "арендуй ВМ, разворачивай приложение" - "опиши задачу AI-агенту, он сделает"
- Аппаратная экономия, конкуренция корпоративных систем за ресурсы, максимизация утилизации



ATAM (Architecture Tradeoff Analysis Method)

Что?

Метод архитектурного анализа для решений, сценарное моделирование
N-дневный процесс с заинтересованными сторонами, архитекторами, бизнесом и пользователями



Mobile Banking

Функции:

- аутентификация,
- просмотр счетов,
- переводы,
- оплата услуг,
- уведомления,
- управление картами

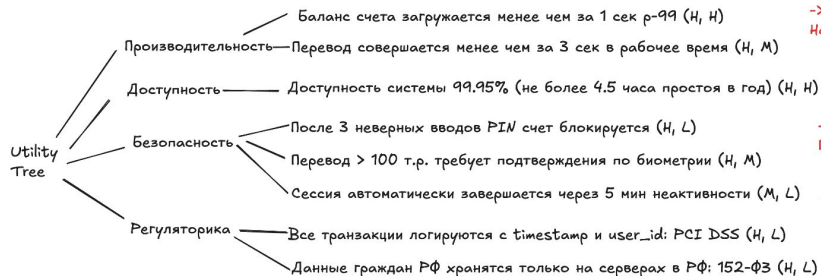
Идея

Архитектурные решения всегда являются **компромиссами между атрибутами качества**

Пример: синхронный вызов дает консистентность, но растет задержка. Кэш дает скорость, но eventual consistency

Категоризация параметров сценариев: (важность для бизнеса, архитектурная сложность / риск)

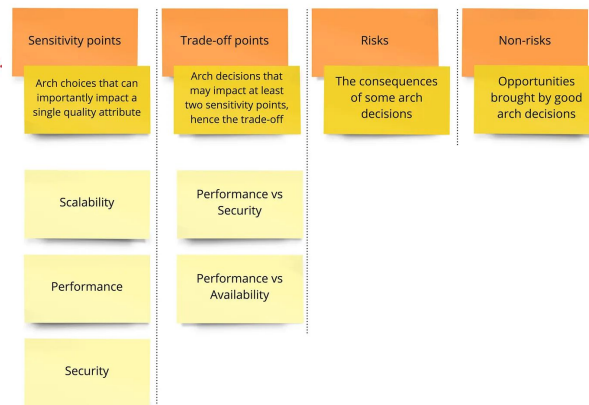
Поиск чувствительных и компромиссных точек и рисков, их формализация и описание вариантов решения



-> Критично для бизнеса + Архитектурно рискованно.
Надо прототипировать и проверять

-> Критично для бизнеса + Архитектурно понятно.
Просто делаем вторым приоритетом

-> Средне для бизнеса + Архитектурно понятно.
Просто делаем третьим приоритетом





SAAM (Software Architecture Analysis Method)

Что?

Метод архитектурного анализа, предшественник ATAM, сценарное моделирование



Mobile Banking

Функции:

- аутентификация,
- просмотр счетов,
- переводы,
- оплата услуг,
- уведомления,
- управление картами

Идея

Выяснить влияние изменений на архитектуру

Взять реальные сценарии изменений, которые точно случатся в системе + оценить влияние

Пример: ЦБ ввел требования об обязательной проверке транзакций на потенциальное мошенничество

Плохая архитектура

Валидация и обработка платежей в разных местах

- TransactionService: обрабатывает переводы между счетами
- PaymentService: обрабатывает оплату услуг
- CardService: обрабатывает операции по картам

Чтобы добавить проверку на фрод

- TransactionService: добавить вызов FraudService
- PaymentService: добавить вызов FraudService
- CardService: добавить вызов FraudService

Риск: один сервис обновили, второй забыли

“Пойдет” архитектура

Все транзакции, независимо от типа, проходят через один TransactionService.

PaymentService и CardService формируют намерение, но не исполняют сами.

Было: PaymentService -> TransactionService -> исполнить

Стало: PaymentService -> TransactionService -> FraudService
-> [одобрено/отклонено] -> исполнить



CBAM (Cost Benefit Analysis Method)

Что?

Метод архитектурного анализа, базирующийся на финансах

Идея

Количественная оценка решений. Обоснование выбора альтернатив через призму финансов

Может быть полезно при выборе из альтернатив и при защите решений бизнесу



Mobile Banking

Функции:

- аутентификация,
- просмотр счетов,
- переводы,
- оплата услуг,
- уведомления,
- управление картами

Переход на EDA

1

Текущее состояние

- Доступность: 99.5%
- 800 RPS в пике
- Т2М фици: 3 дня

2

Стоимость перехода

- Dev (300 т.р./М) : 5 инж. x 4 М = 20 ч/М
 - QA (200 т.р./М) : 2 инж. x 2 М = 4 ч/М
 - SRE (250 т.р./М): 1 инж. x 12 М = 12 ч/М
 - Инфра: Kafka-кластер = 120 т.р./год
-
- Единоразово: $300 * 20 + 200 * 4 = 6,8$ млн. р.
 - Каждый год: $250 * 12 + 120 \approx 3,1$ млн. р.

3

Выгода по атрибутам

- Доступность 99.5% -> 99.95%
(экономия 3 млн.р./год на простоях)
- 800 RPS -> 5 000 RPS
(дополучаем выручку в 5 млн.р./год)
- Т2М фици: 2 дня
(при том же бэклоге сокращаем ТСО на 200 т.р. ежемесячно)

4

ROI

Начальные инвестиции: 6.8 млн.р.
Выгода в год: 8.2 млн.р.
Траты в год: 3.1 млн.р.
Дельта: +5.1 млн.р.
Окупаемость: $6.8/5.1 = 1,3$ года



RFC (request for comments) + ADR (architecture decision record)

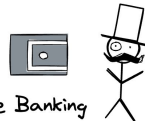
Что?

Форматы описания (документы) архитектурных решений

Идея

Не хватает индексации, формализации и систематизации принятых решений по архитектуре

Выход - создание упорядоченного хранилища архитектурных обсуждений и решений



Mobile Banking

Функции:

- аутентификация,
- просмотр счетов,
- переводы,
- оплата услуг,
- уведомления,
- управление картами

RFC

RFC-001: Стратегия управления токенами

Контекст

При НТ обнаружили: валидация токенов через PostgreSQL создает бутleneck при > 10 тыс. одновременных пользователей.

Предлагаемое решение

Кэшировать access token. Валидация access token будет проходить чаще без обращения к БД.

Альтернативы

А. Оставить как есть, но добавить read-only реплику. Отклоняем: не решает проблему, добавляет операционную сложность. При 40 000 пользователей реплика тоже станет бутleneckом.

Б. JWT без механизма инвалидации. Отклоняем: неприемлемо для банка. При краже токена юзер уязвим до истечения TTL.

Трейдоффы

Выигрываем: р99 задержка: 1.8 сек -> ожидаем < 1 сек, горизонтальное масштабирование без sticky sessions

Теряем: новая зависимость: Redis как критичный компонент, выше косты на инфраструктуру

Открытые вопросы

1. Как обрабатывать одновременный refresh acces token с двух устройств?
2. Что если Redis упадет? Как организовать отказоустойчивость Redis?

ADR

ADR-001: Стратегия управления токенами

Контекст

При НТ обнаружили: валидация токенов через PostgreSQL создает бутleneck при > 10 тыс. одновременных пользователей.

Принятое решение

Кэшировать access token. Валидация access token проходит по такому механизму с такими-то параметрами.

Причины выбора

1. Stateless валидация убирает бутleneck на БД
2. Есть мгновенная инвалидация на Redis
3. Не умеем работать с sticky sessions (но лучше так не писать)

Последствия

ОК: действительно, видим, что р99 задержка снизилась до 300 мс. Штатно работает при 50 тыс. сессий.

Не ОК: Redis стал критичным компонентом, его падение означает невозможность функций авторизации

Отклоненные альтернативы

1. А и Б - почему? Подробности в RFC-001.



Fitness Function

Что?

Метод архитектурного анализа, встроенного в непрерывные процессы (CI/CD)
Функции пригодности измеряют степень соответствия архитектуры поставленной цели

Идея

Архитектура постоянно эволюционирует, значит нужен постоянный мониторинг архитектурного здоровья

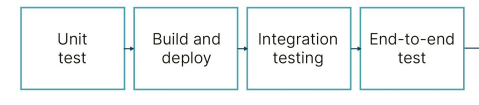
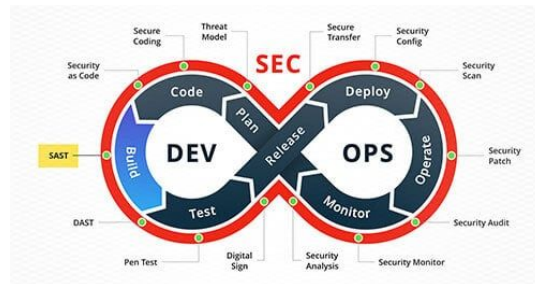


Mobile Banking

Функции:

- аутентификация,
- просмотр счетов,
- переводы,
- оплата услуг,
- уведомления,
- управление картами

```
1 def test_balance_endpoint_performance():
2     run_load_test(
3         endpoint="/api/v1/account/balance",
4         rps=50_000,
5         duration_sec=60
6     )
7     assert p99_latency_ms < 1000
8     assert error_rate < 0.001
9
10 def test_no_pii_in_logs():
11     logs = get_last_hour_logs()
12     for log in logs:
13         assert not contains_card_number(log), \
14             f"Номер карты найден в логах: {log['service']}"
15         assert not contains_passport(log), \
16             f"Паспортные данные найдены в логах: {log['service']}"
17
```





Выводы

Архитектурный анализ - это база

Если грамотно использовать методы, фреймворки, инструменты анализа - архитектуре всегда будет ОК

Методы анализа работают сообща

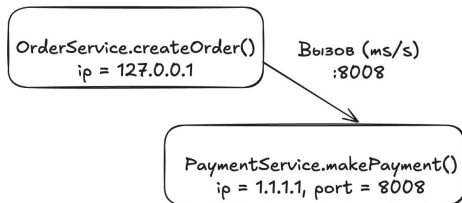
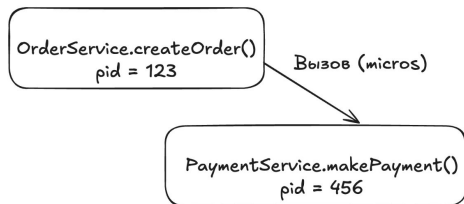
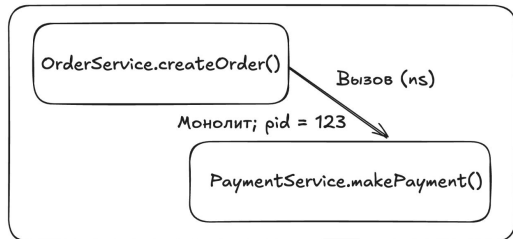
Разные инструменты для разных моментов жизни системы: ATAM - до начала, SAAM - когда нужно проверить готовность к изменениям, CBAM - когда нужно говорить с бизнесом его языком, Fitness Functions, RFC, ADR - постоянно

Общие тезисы

1. ATAM без честных приоритетов бесполезен. Если все сценарии High, то ничто не High. Utility Tree работает только тогда, когда команда готова сказать: "этот сценарий менее важен, чем другой"
2. SAAM - самый быстрый способ обнаружить плохие границы, достаточно спросить: "что мы будем менять, когда придет это требование?"
3. RFC пишут на этапе набросков решения. ADR пишут после принятия архитектурного решения. Если писать ADR после деплоя - это просто документирование факта, а не мышления
4. Кто внедряет фитнес функции, тот молодец



Межсервисные взаимодействия [0]



Latency Numbers Every Programmer Should Know

■ 1 ns

■ L1 cache reference: 0.5 ns

■ Branch mispredict: 5 ns

■ L2 cache reference: 7 ns

■ Mutex lock/unlock: 25 ns

■ = 100 ns

■ Main memory reference: 100 ns

■ = 1 μ s

■ Compress 1 KB with Zippy: 3 μ s

■ = 10 μ s

■ Send 1 KB over 1 Gbps network: 10 μ s

■ SSD random read (1 GB/s SSD): 150 μ s

■ Read 1 MB sequentially from memory: 250 μ s

■ Round trip in same datacenter: 500 μ s

■ = 1 ns

■ Read 1 MB sequentially from SSD: 1 ns

■ Disk seek: 10 ns

■ Read 1 MB sequentially from disk: 20 ns

■ Packet roundtrip CA to Netherlands: 150 ns

Source: <https://gist.github.com/2841832>



Межсервисные взаимодействия [1]

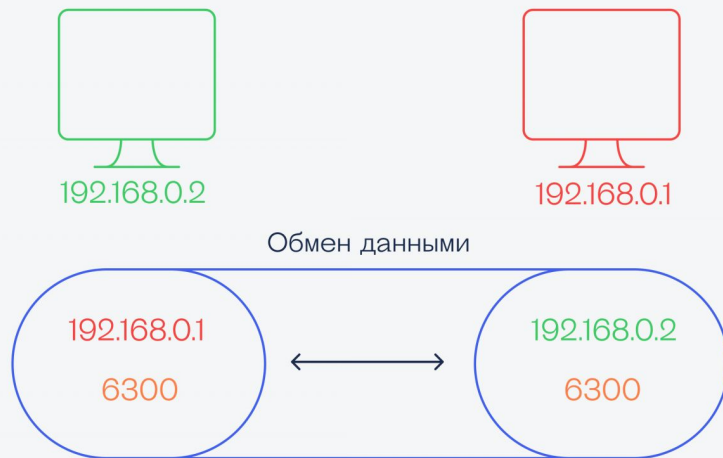
Сокеты и порты

- У процессов есть **сокеты**
- Сокет - точка входа в процесс, к которой можно подключиться по IP-адресу и порту
- Процесс резервирует порт, порт - индекс/адрес под сокет
- Соединение - подключение между двумя сокетами (двунаправленный канал)
- На один порт может быть только один слушатель, но он может обслуживать много входящих соединений
- Любое кол-во соединений может быть на одном порту, каждое из которых приводит к новому сокету

Отправитель
и получатель данных

Между ними открыты
сокеты, в которых есть:

- IP адрес другой стороны
- Один и тот же порт

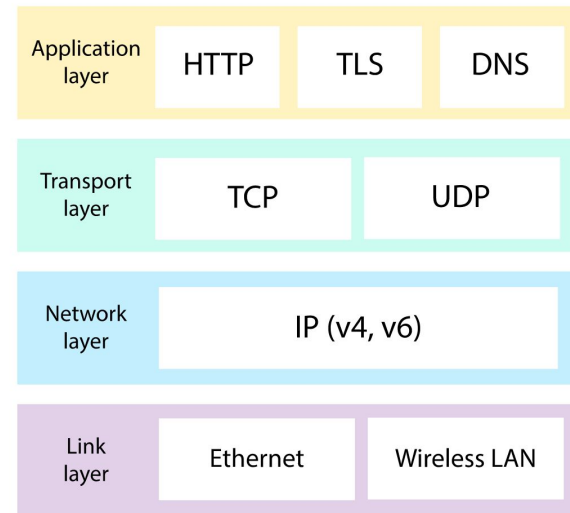




Межсервисные взаимодействия [2]

ТСР и HTTP

- **ТСР** - протокол транспортного уровня, который гарантирует, что:
 - пакеты дойдут или отправитель узнает об ошибке;
 - пакеты придут в том же порядке, в котором отправлены;
 - данные не искажаются в пути
- Над ТСР **HTTP** - протокол прикладного уровня для передачи гипертекстовых данных:
 - HTTP/1.1 - одно соединение, один запрос за раз (или несколько соединений параллельно, неэффективно)
 - HTTP/2 - одно соединение, много запросов параллельно (multiplexing). Бинарный протокол, сжатие заголовков. На нем работает gRPC
 - HTTP/3 - поверх UDP (QUIC). Повышает скорость, надежность и безопасность соединения, особенно в нестабильных сетях





Межсервисные взаимодействия [3]

Связь интеграций и атрибутов качества

- Производительность:
 - синхронный вызов добавляет задержку на каждый hop
 - неправильный протокол - лишний overhead на сериализацию
- Отказоустойчивость:
 - упал один сервис - как это влияет на остальные?
 - синхронная цепочка из 5 сервисов = доступность перемножается
- Безопасность:
 - каждая интеграция - потенциальная точка атаки
 - кто может вызывать этот сервис? как аутентифицировать?
- Наблюдаемость:
 - как понять, где упало в цепочке вызовов?

Синхронное

- + Простая реализация
- + Простое тестирование
- + Простая отладка
- Снижение доступности
- Ожидание ответа
- Высокая связанность

Асинхронное

- + Высокая доступность
- + Снижение времени ожидания
- + Слабая связанность
- Сложная реализация
- Сложное тестирование
- Сложная отладка
- Eventual Consistency

Синхронная интеграция

Плюсы

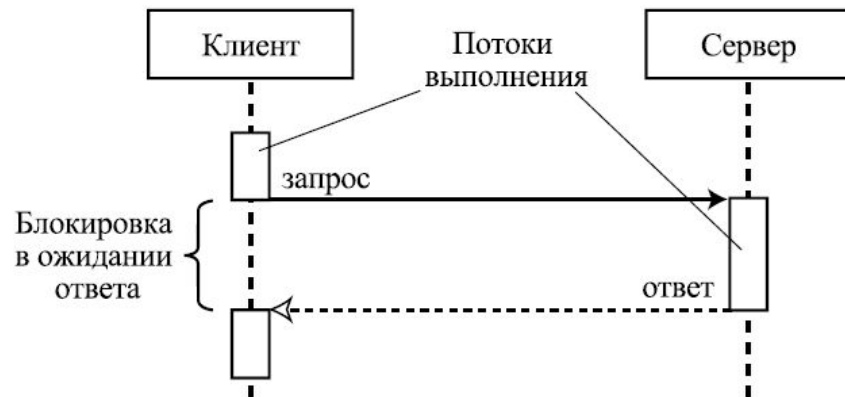
- Простая модель: запрос -> ответ, линейный флоу
- Немедленный результат: сразу определяем успех/ошибку
- Легко дебажить, стектрейс показывает весь путь
- Естественно для запросов данных и пользовательских операций

Минусы

- Оба сервиса должны быть живы одновременно
- Занимает ресурсы: поток и соединение заняты всё время ожидания
- Потенциальные каскадные сбои

Атрибуты качества

- **Отказоустойчивость:** во время запроса может умереть как вызывающий, так и вызываемый
- **Масштабирование:** один экземпляр вызываемого вызывает один экземпляр вызывающего
- **Производительность:** таймауты





Асинхронная интеграция [0]

Плюсы

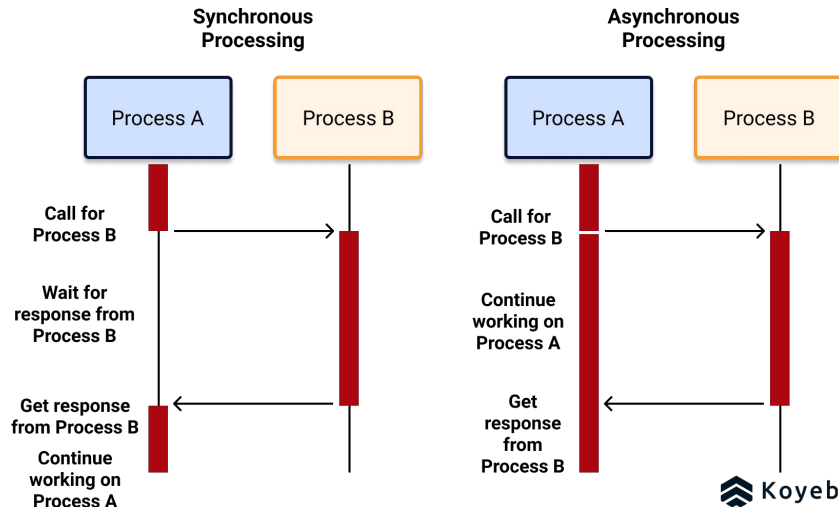
- Producer работает, даже если consumer лежит
- Независимое масштабирование: добавляешь consumer под нагрузку
- Естественный буфер: пиковая нагрузка буферизуется в брокере
- Легко добавить нового consumer без изменения producer

Минусы

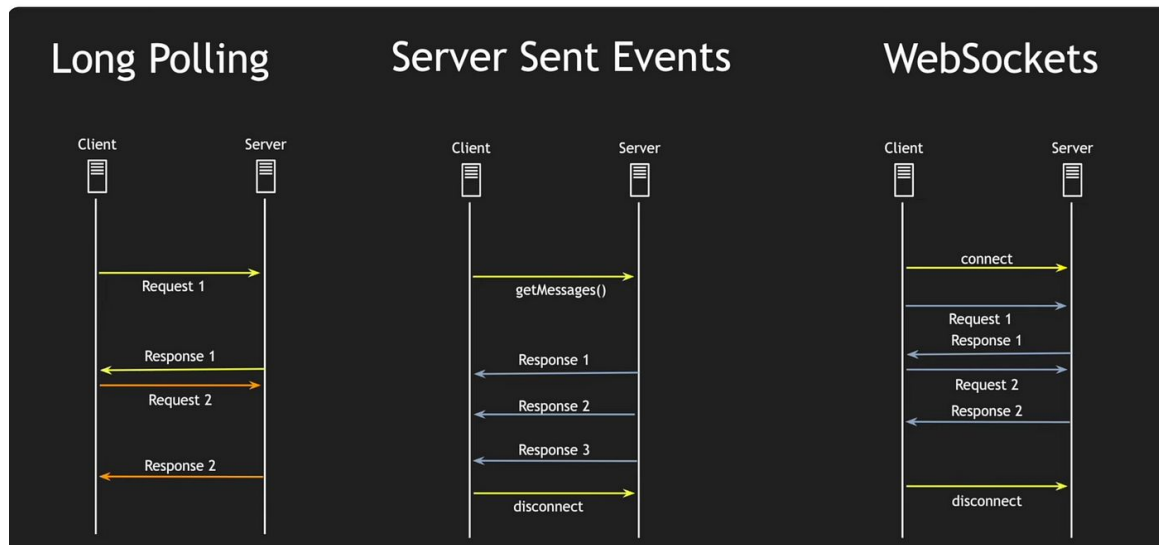
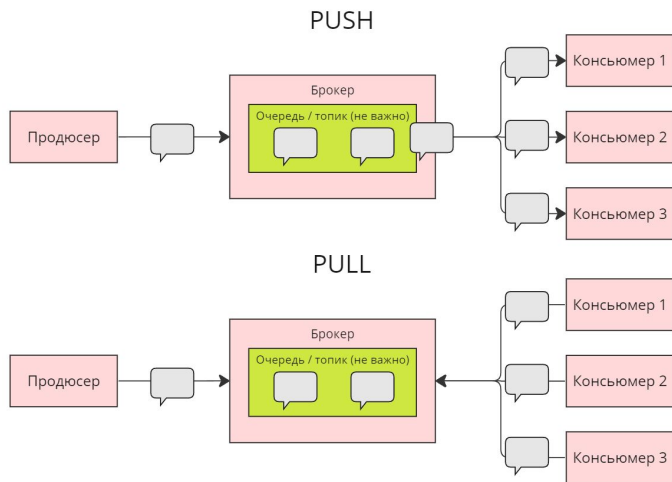
- Eventual consistency: результат будет, но когда-то
- Сложнее дебажить: нет линейного трейса, событие "где-то там"
- Обязательная обработка идемпотентности: сообщение может прийти дважды
- Инфраструктурный overhead: брокер/очередь еще один компонент

Примеры

- Message Broker / MQ
- WebSocket
- SSE (Server-Site Event)
- Long Polling
- Webhook



Асинхронная интеграция [1]



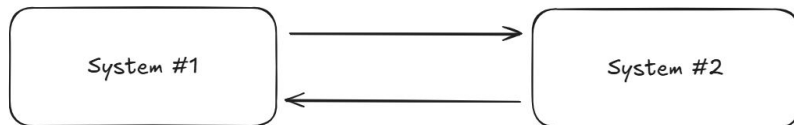


Алгоритм выбора

Сценарии

1. Получить данные на Frontend для отрисовки корзины товаров
 - а. Синхронное: хотим увидеть товары здесь и сейчас
2. Требуется гарантированная доставка документов
 - а. Асинхронное: можно подумать над ретраями, но лучше Kafka + DLQ
3. Генерация отчета
 - а. Асинхронное: долгая операция, сгенерироваться может и позже
4. Бизнес хочет, чтобы в браузере рендерилось уведомление (работа колокольчика)
 - а. Асинхронное: вебхуки или sse
5. Поточковая обработка данных, например, логирование
 - а. Асинхронное: не требуют мгновенной обработки
6. Перевод денег в онлайн-банке
 - а. Синхронное
7. Бронируем билеты в отель
 - а. Синхронное: нельзя допустить бронь одного места 2 раза

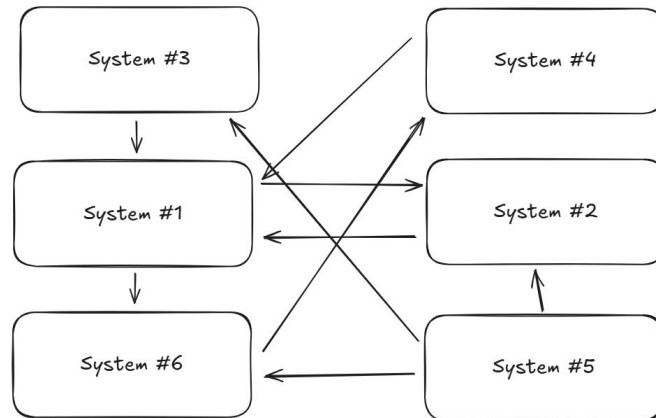
P2P (точка-точка, point-to-point)



N сервисов = до $N \times (N-1)/2$ прямых связей

2 сервиса = $2 \times 1/2 = 1$ связь

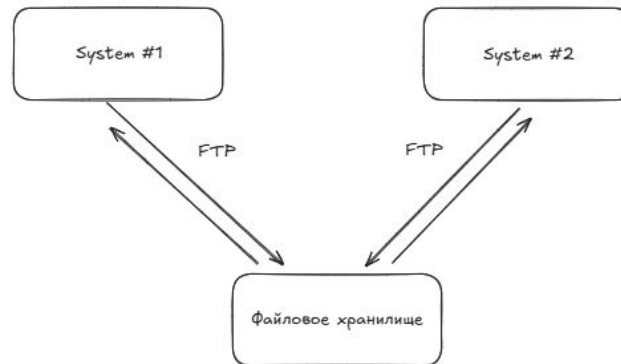
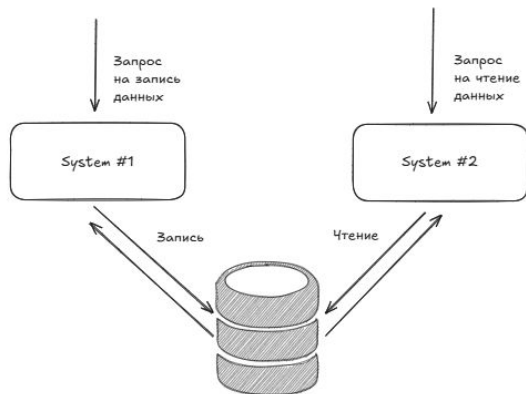
5 сервисов = $5 \times 4/2 = 10$ связей



Особенности

- Каждый сервис знает адреса других
- При изменении API в одном сервисе требуется обновление во всех вызывающих сервисах
- Нет центральной точки контроля, наблюдаемости, безопасности
- Спагетти-интеграция

Общая база данных и FTP

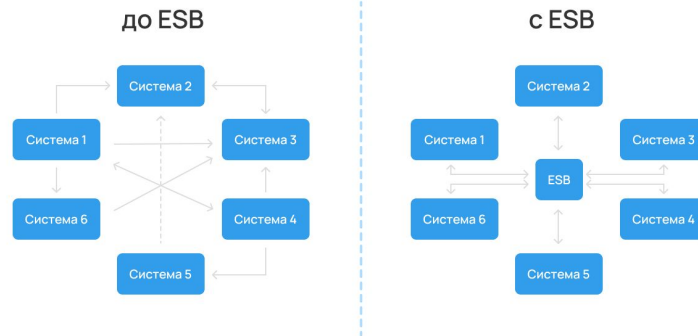


Особенности (БД)

- Плюс: не нужна новая инфраструктура, все просто и понятно
- Минусы:
 - Схема БД становится публичным контрактом между сервисами
 - Нет четкого владельца данных
 - Нагрузка одного сервиса влияет на другой
 - Невозможно масштабировать сервисы независимо

Особенности (FTP)

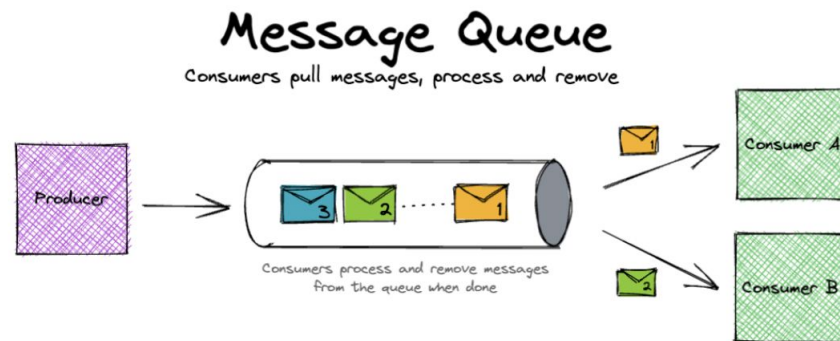
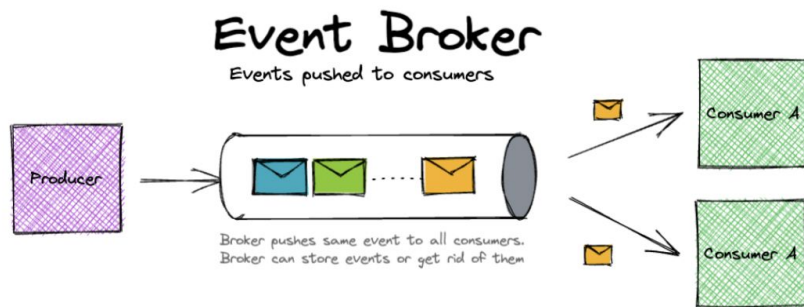
- Хорошо работает для больших объемов данных, пакетной обработки, интеграций с системами "умею только в файлы"
- Минусы:
 - Задержка: не real-time, а batch (час, день)
 - Обработка ошибок сложна: файл прочитан наполовину, что делать?
 - Формат файла: жесткий контракт, менять болезненно
 - Нет подтверждения обработки



Особенности

- Центральная шина, через которую идут все интеграции
- ESB умеет: маршрутизировать сообщения, трансформировать форматы (SOAP -> REST, XML -> JSON), оркестрировать бизнес-процессы, централизовать безопасность и мониторинг
- На практике:
 - ESB становился god object: вся бизнес-логика интеграций постепенно переезжала в шину
 - Единая точка отказа: все сообщения через один компонент
 - Дорого: лицензии IBM, Oracle, MuleSoft + поддержка
 - "Умная труба": вместо того чтобы быть тупым транспортом, ESB знал о бизнесе слишком много

Брокер сообщений / очередь сообщений



Особенности

- "Умная труба" для передачи сообщений, но без бизнес-логики
- Message Queue (RabbitMQ / ActiveMQ):
 - сообщение -> один получатель, потом удаляется
 - конкурирующие потребители делят нагрузку
- Message Broker (Kafka):
 - сообщение -> несколько получателей независимо
 - сообщения хранятся на брокере
 - потребитель сам отслеживает свою позицию

MQ

- Один producer кладет сообщение - один consumer его забирает. После обработки сообщение исчезает
- Каждое сообщение обрабатывается ровно одним consumer

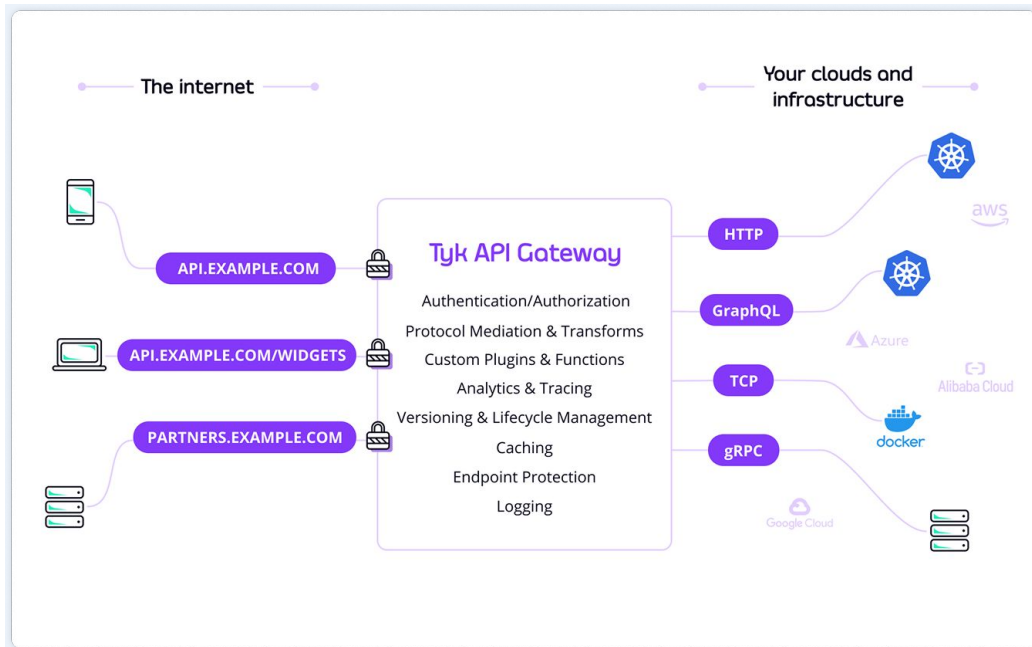
Broker

- Модель publish-subscribe. Producer публикует сообщение в топик - все подписчики получают его независимо
- Сообщение не исчезает после прочтения. Каждый consumer group читает независимо, отслеживая свою позицию (offset)

API GATEWAY

Особенности

- Появился как ответ на вопрос: как внешний клиент (мобильное приложение, система-потребитель) работает с десятками внутренних сервисов?
- Шлюз централизует: аутентификацию и авторизацию запросов, rate limiting и throttling (квоты), SSL терминацию, маршрутизацию, агрегацию нескольких вызовов в один, балансировку нагрузки, трансформацию данных, шифрование, паттерны отказоустойчивости, и т.д.
- Проблемы
 - SPOF (single point of failure)
 - Соблазн добавить бизнес-логику
 - Может быть bottleneck





НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ